Enterprise DevSecOps Security Lab

Kubernetes Security Assessment

Detailed assessment of cluster posture, workload hardening, RBAC, NetworkPolicy, Pod Security, manifest scanning, and CIS Benchmark alignment.

Field	Value
Document Type	Assessment Report
Author	Jagriti Banerjee
Project Scope	Private Kubernetes DevSecOps lab using Docker, Kind, Calico, Trivy, Falco, OPA/Gatekeeper, ArgoCD, and kube-bench
Audience	Security engineers, DevSecOps reviewers, recruiters, and technical interviewers
Classification	Private lab / portfolio evidence

Reviewer snapshot

Review Dimension	Included in Document
Technical depth	Control rationale, validation commands, expected results, risk interpretation, and remediation guidance.
Evidence model	Screenshot references, manifest fields, scanner outputs, and repeatable kubectl commands.
Professional use	Can be used as a GitHub portfolio artifact, interview talking point, and internal-style security record.
CIS alignment	Includes a dedicated CIS Kubernetes Benchmark section with lab-context interpretation.

Scope statement

This document is based on controlled, authorized testing performed inside a private lab environment. It does not represent a production certification and does not include public or third-party targets. The focus is professional evidence, repeatable validation, and security engineering reasoning.

Spacing and readability note

The document is intentionally laid out with compact paragraphs, split tables, and dedicated evidence pages so that detail is preserved without crowded pages or large blank gaps.

Document coverage

Coverage Area	Included
Assessment narrative	Executive summary, scope, methodology, technical analysis, and validation interpretation.
Evidence references	Relevant screenshots, command outputs, reports, and expected retest outcomes.
Remediation planning	Priority-based actions, production notes, and follow-up recommendations.
Benchmark context	CIS Kubernetes Benchmark alignment and lab-specific interpretation.

1. Executive Summary

This assessment reviews the Kubernetes security posture implemented for the Enterprise DevSecOps project. The review confirms that the demo application is deployed using a hardened namespace, a dedicated ServiceAccount, least-privilege RBAC, strict pod security settings, Calico-enforced NetworkPolicies, and manifest scanning. It also documents residual risk and production-readiness improvements.

Baseline Area	Lab Implementation
Cluster	Kind Kubernetes running inside Ubuntu-Endpoint private VM.
Network plugin	Calico installed so NetworkPolicy behavior can be validated.
Application	Hardened Flask demo app deployed as demo-app in devsecops namespace.
Security controls	RBAC, NetworkPolicy, Pod Security, hardened securityContext, Trivy, kube-bench, Falco evidence.
Evidence	Screenshots and reports under evidence/screenshots and security/reports.

2. Assessment Objectives

- Verify that Kubernetes workload identity is separated from the default ServiceAccount.
- Confirm that namespace-level Pod Security restrictions are present and testable.
- Confirm that pod and container security contexts reduce runtime privilege.
- Validate that network access is restricted using a default-deny model with explicit allow rules.
- Review RBAC permissions to ensure the application cannot access secrets or cluster-wide resources.
- Include CIS Kubernetes Benchmark context and kube-bench evidence as part of the security posture story.
- Document remediation, retest criteria, and production-hardening recommendations.

3. Methodology

3.1 Review model

The assessment combined static YAML review, live kubectl validation, scanner output review, and screenshot evidence. A control was considered validated only when it had a manifest setting, a live command result, or a captured screenshot proving that the expected state was present.

3.2 Commands used during validation

```
kubectl get nodes -o wide
kubectl get ns devsecops --show-labels
kubectl get sa,role,rolebinding -n devsecops
kubectl auth can-i get secrets --as=system:serviceaccount:devsecops:demo-app-sa -n devsecops
kubectl get networkpolicy -n devsecops
kubectl get pod -n devsecops -l app=demo-app -o jsonpath='{.items[0].spec.securityContext}'
trivy config k8s/base
kubectl logs job/kube-bench || true
```

3.3 Evidence interpretation

These commands validate both configuration and behavior. The assessment does not treat YAML presence as sufficient evidence; it validates the live cluster state, expected denials, and scanner outputs before considering the control implemented.

4. Control Review Matrix

Control Area	Expected Security State	Observed / Evidence	Result
Namespace isolation	Workloads run in a dedicated namespace with restricted Pod Security labels.	devsecops namespace and label evidence captured.	Pass
ServiceAccount isolation	Workload uses demo-app-sa instead of default SA.	Deployment references demo-app-sa; RBAC objects created.	Pass
RBAC	Application can read only required ConfigMaps and cannot read secrets or list nodes.	kubectl auth can-i confirms allow/deny behavior.	Pass
Runtime hardening	Non-root user, no privilege escalation, dropped capabilities, RuntimeDefault seccomp.	SecurityContext evidence captured from live pod.	Pass
Filesystem hardening	Root filesystem is read-only; only /tmp writable through emptyDir.	Write-to-/app fails; write-to-/tmp succeeds.	Pass
Network segmentation	Default deny blocks untrusted pods; explicit rules allow trusted clients.	Allowed and blocked curl pod tests captured.	Pass
Manifest scanning	Kubernetes manifests are scanned for critical misconfiguration.	Trivy config scan evidence captured.	Pass
CIS benchmark review	kube-bench output is captured and interpreted with Kind limitations.	kube-bench screenshot and report evidence captured.	Completed

5. Detailed Observations

5.1 Strengths

- The deployment uses non-root execution and a fixed UID/GID, reducing impact if application code is exploited.
- The ServiceAccount token is not automatically mounted, reducing Kubernetes API token exposure inside the pod.
- NetworkPolicy validation demonstrates both positive and negative tests rather than only applying YAML.
- The project includes runtime detection and admission-style blocking evidence, which improves the security narrative.
- The lab documents not only controls but also retest commands, which is important for real-world remediation closure.

5.2 Residual risk

- Kind is not a production cluster; control plane and node configuration findings must be revalidated on the target distribution.
- imagePullPolicy: Never and local image loading are appropriate for a private lab but should be replaced by signed registry images in production.
- CIS Benchmark evidence should be treated as secure configuration guidance, not a guarantee that every workload is secure.
- Admission enforcement should be expanded to block unsigned images and missing labels automatically.

6. CIS Kubernetes Benchmark Section

CIS describes the Kubernetes Benchmark as consensus-based secure configuration guidance for Kubernetes. In this lab, CIS Benchmark alignment is used as an assessment reference and learning framework. Because the cluster is a Kind-based private lab rather than a production kubeadm or managed service environment, benchmark results are interpreted with environment context. The expected professional practice is to document PASS, FAIL, WARN, and not-applicable outcomes; assign owners to applicable failures; retest after remediation; and keep benchmark artifacts in the evidence folder.

CIS Review Area	Lab Evidence / Interpretation	Follow-up
Access control	RBAC reviewed through kubectl auth can-i commands.	Keep least-privilege checks in CI/retest process.
Workload security	Pod Security restricted labels and hardened securityContext applied.	Enforce with admission policies across all namespaces.
Network controls	Calico NetworkPolicy evidence validates traffic restrictions.	Apply namespace-wide default-deny policy pattern.
Auditability	Screenshots and reports collected for benchmark and manifest scan evidence.	Create issue tracker mapping for failed or warning items.

7. Evidence Highlights

8. Remediation Roadmap

Priority	Recommendation	Reason
P1	Keep critical security gates required in pull requests.	Prevents critical dependency, container, secret, Dockerfile, and Kubernetes misconfiguration regressions.
P1	Enforce signed images using Sigstore Policy Controller or Kyverno verifyImages.	Converts supply-chain evidence into admission-time prevention.
P2	Expand CIS benchmark review into a tracked remediation register.	Improves auditability and closure.
P2	Add scheduled Trivy/kube-bench scans.	Detects drift and newly disclosed vulnerabilities.
P3	Add cloud-provider managed Kubernetes assessment later.	Shows production-readiness beyond Kind lab scope.

Quality Assurance and Evidence Preservation

QA Item	Required Practice
Evidence naming	Use stable screenshot names and link each item to a control or retest step.
Command repeatability	Store commands in documentation so another reviewer can reproduce validation.
Exception handling	Any accepted risk should include owner, reason, expiry date, and compensating control.
Retest discipline	Do not close a remediation item until the expected negative and positive tests both pass.
CI/CD linkage	Where possible, convert manual validation into security gates or scheduled checks.

Reviewer notes for this Kubernetes security area

This document intentionally combines implementation evidence with risk interpretation. The objective is to show not only that commands were executed, but also why each control matters, how it reduces attack surface, and how the result should be retested after future changes.

References and Standards Basis

Reference	Use in this Document
CIS Kubernetes Benchmark	CIS describes its Kubernetes Benchmark as a product of a community consensus process with secure configuration guidelines for Kubernetes.
Kubernetes RBAC documentation	Kubernetes RBAC uses Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects to manage authorization.
Kubernetes NetworkPolicy documentation	NetworkPolicy governs how pods communicate with each other and with network endpoints.
Kubernetes Pod Security Standards	The restricted profile is intended for strongly hardened workload execution.

Reference URLs used during document preparation: https://www.cisecurity.org/benchmark/kubernetes, https://kubernetes.io/docs/reference/access-authn-authz/rbac/, https://kubernetes.io/docs/concepts/services-networking/network-policies/,

and https://kubernetes.io/docs/concepts/security/pod-security-standards/.